Actividad 5: Modelado de Datos con Estructuras (Struct) y Punteros

Leon Melgarejo

EEST N.º 1 - "Eduardo Ader" Vicente López

Curso: Laboratorio de Programación 5° año 3° - Grupos A-B

Profesor: Mansilla Muñoz York Elías

Fecha: 25 de Septiembre de 2026

Introducción

El presente informe académico documenta los resultados y fundamentos teóricos de la Actividad 5 de la materia Laboratorio de Programación. Esta unidad se enfoca en el desarrollo de metodologías avanzadas de gestión de memoria y modularización utilizando el lenguaje de programación C++.

El propósito central es asimilar el concepto de modelado de entidades complejas del mundo real, orientándolo de forma directa hacia la arquitectura técnica necesaria en el Proyecto Integrador Anual (P.I.A.) para la Expotécnica de la institución. A lo largo del documento, se analizan los paradigmas de organización heterogénea de datos, la manipulación de direcciones físicas de memoria mediante punteros y los mecanismos obligatorios de control y saneamiento en los flujos de entrada de la consola estándar.

Desarrollo

1. Entornos de Desarrollo y Compilación

Para la ejecución y validación de esta actividad se contemplan dos entornos tecnológicos diferenciados según la disponibilidad técnica del laboratorio escolar:

> Entorno Local (VS Code + MSYS2): Empleado en terminales con permisos de administrador. Permite compilar mediante la consola integrada de PowerShell empleando la herramienta g++.

> Entorno Remoto (OnlineGDB): Utilizado de forma obligatoria en las computadoras de la escuela que posean restricciones de ejecución o bloqueos de políticas de seguridad locales.

El comando formal de compilación estructurado desde la terminal es:

powershell
g++ src/main.cpp -o src/sistemaproyecto.exe

Usa el código con precaución.

Y su posterior comando de ejecución en la plataforma Windows es:

powershell
.\src\sistemaproyecto.exe

2. Estructura Heterogénea de Datos (struct)

Una estructura (struct) se define técnicamente como una colección de variables de distintos tipos lógicos que se agrupan bajo un mismo identificador común en bloques contiguos de la memoria RAM. A diferencia de las variables aisladas, la ventaja crítica del struct radica en la **cohesión y abstracción**. Permite encapsular las diferentes variables y propiedades de un componente físico real (como sensores, módulos de hardware o usuarios) dentro de un único tipo de dato personalizado.

El prototipo del modelo diseñado para el software del proyecto integrador se declara de la siguiente manera:

cpp

```
/*
   ================================================================
   Materia: Laboratorio de Programación (LPR) — 5°
   Archivo: src/main.cpp
   Actividad 5: Modelado de datos con Structs y Punteros
   ================================================================
*/

#include <iostream>
#include <cstring>

using namespace std;

// 1. DEFINICIÓN DE LA ESTRUCTURA (Adaptada a un ejemplo de Proyecto Integrador: Sensor Ambiental)
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica; // En este ejemplo representará la temperatura en °C o humedad
```

```
};

// Prototipo de función con pasaje por dirección mediante puntero
void cargarDatos(EntidadProyecto* ptr);

int main() {
    // CAMBIAR: Coloca tu Nombre y Apellido real en la inicialización de seguridad
    EntidadProyecto miEntidad = {0, "Vacio - Juan Perez", 0.0f};

    cout << "======================================================" << endl;
    // CAMBIAR: Coloca tu Nombre y Apellido real en el encabezado de la consola
    cout << "  MODELADO STRUCT - ESTUDIANTE: Juan Perez " << endl;
    cout << "======================================================" << endl;

    // Invocación enviando la dirección física de memoria con '&'
    cargarDatos(&miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
    cout << "ID Registrado:     " << miEntidad.id << endl;
    cout << "Nombre Registrado: " << miEntidad.nombre << endl;
    cout << "Metrica Guardada:  " << miEntidad.metrica << endl;
    cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
    cout << "======================================================" << endl;

    return 0; // Código 0: Finalización exitosa
```

```
}

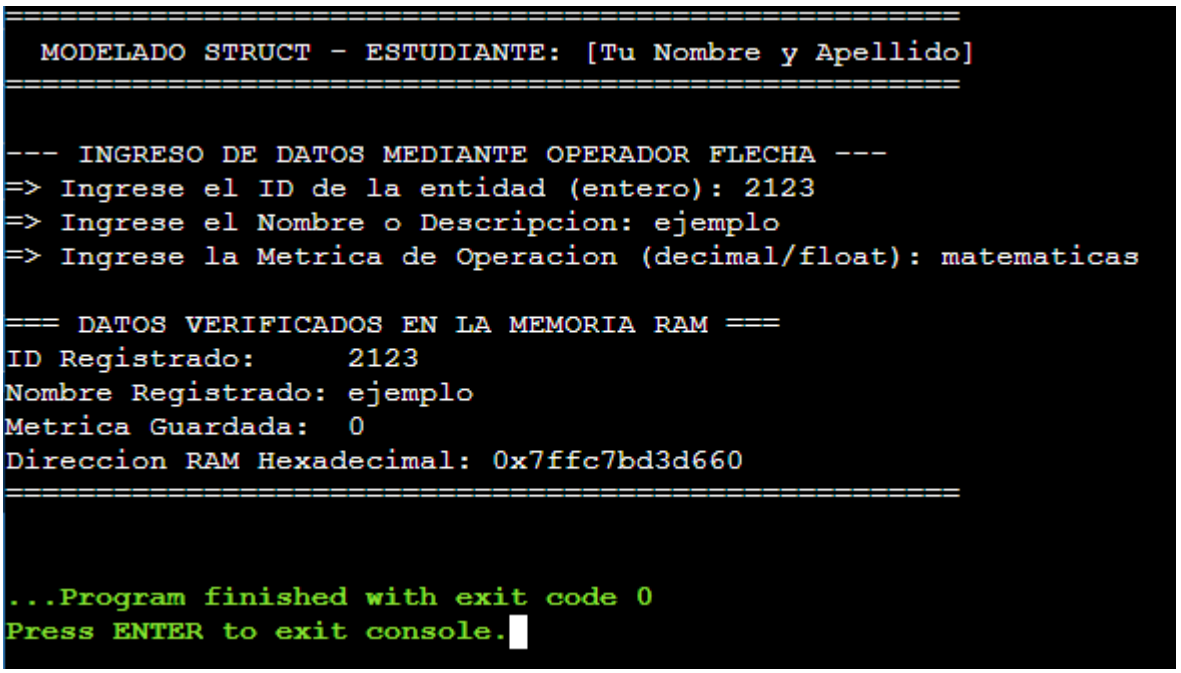
void cargarDatos(EntidadProyecto* ptr) {
    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;

    cout << "=> Ingrese el ID de la entidad (entero): ";
    cin >> ptr->id;

    // Limpieza del buffer obligatoria antes de cin.getline
    cin.ignore();

    cout << "=> Ingrese el Nombre o Descripcion (Ej: Sensor_DHT22): ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica de Operacion (Ej de lectura/float): ";
    cin >> ptr->metrica;
}
```



3. Gestión de Memoria y Punteros

El programa utiliza dos herramientas de direccionamiento lógico fundamentales para optimizar el rendimiento del hardware:

> **Pasaje por Referencia (Dirección):** Se invoca la función pasando la dirección física de la variable (&miEntidad). Esto optimiza drásticamente el uso de la memoria RAM debido a que no duplica el peso del objeto en el *Stack*, sino que trabaja directamente sobre la dirección original de la variable mapeada en el mapa hexadecimal.

> **Operador Flecha (->):** Es la sintaxis explícita de C++ utilizada cuando se accede a los campos internos de un struct a través de un puntero desreferenciado. El compilador no permite la notación de punto tradicional (ptr.id) cuando la variable es un puntero, dado que el punto opera estrictamente sobre instancias directas, mientras que el operador flecha realiza implícitamente la desreferenciación del puntero antes de acceder al miembro ((*ptr).id).

4. Ciclo de Ejecución de Memoria de la Actividad

De acuerdo con las trazas analizadas en el diagrama de control de la guía, la mutación de la memoria RAM sigue el siguiente comportamiento secuencial:

1. Antes de la línea 6: Las variables num1, num2 y los punteros ptr1 y ptr2 contienen basura de memoria (????).
2. Tras la línea 7: Se inicializa ptr1 apuntando a la dirección de num1 (100) y ptr2 apuntando a la dirección de num2 (104).

3. Tras la línea 10: Se asignan valores numéricos directos a las celdas mapeadas; num1 pasa a valer 10 y num2 toma el valor de 20.
4. Tras la línea 13: El puntero ptr2 se desconecta de la variable reasignando su valor a NULL, garantizando la seguridad del flujo de datos.

5. Control de Flujos de Entrada de Datos

Durante la carga de caracteres por teclado mediante consola, el sistema almacena el código de escape de la tecla Enter (\n) en una sección temporal denominada Buffer de Entrada.

Si se omite el método de saneamiento cin.ignore() antes de llamar a un capturador de cadenas complejas como cin.getline(), la instrucción lee el carácter residual \n remanente de las lecturas numéricas previas (cin >>). Esto provoca un comportamiento erróneo crítico: la terminal interpreta que el usuario ha enviado una cadena vacía y salta automáticamente la entrada de texto sin permitir que se digite el valor real.

Conclusión

La implementación de la Actividad 5 demuestra la importancia de la correcta administración de recursos a nivel de software. El uso de tipos estructurados (struct) proporciona el nivel de abstracción necesario para organizar la telemetría e información de hardware en el Proyecto Integrador Anual de manera limpia y modular.

Asimismo, comprender la mecánica interna del pasaje de parámetros por dirección de memoria e implementar rutinas estrictas de limpieza en el buffer de entrada reduce significativamente la aparición de fallas lógicas imprevistas y fugas de procesamiento. Estas prácticas asientan un estándar técnico robusto y profesional para la posterior entrega de código auditable respaldado en repositorios de GitHub.

Referencias

Cplusplus.com. (2026). *Data structures*. Estructuras en C++ - Guía rápida y oficial. cplusplus.com

EEST N.º 1 "Eduardo Ader". (2026). *Actividad 5: Modelado de datos con estructuras (struct) y punteros*. Laboratorio de Programación, 5° año 3°.

GitBook. (2026). *Git Book en Español*. Herramientas de control de versiones, commits y ramas de desarrollo. git-scm.com

https://github.com/letan-doto07/actividad5-modelado-de-datos-con-estructuras